EDA --> Exploratory Data Analysis

EDA is foundation of machine learning EDA technique will hepls to convert raw data to clean data

1. Variable Identification
2. Univariate Analysis
3. Bivariate Analysis
4. Imputation Technique (Transformer)
5. Missing Value treatment
6. Outlier Treatment
7. New Variable Creation

Convert Raw Data To Clean Data

```
import pandas as pd
```

```
pd.__version__
```

```
'2.3.3'
```

```
emp = pd.read_excel(r"C:\Users\HP\Downloads\Rawdata.xlsx")
```

```
emp
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

```
id(emp)
```

```
1682286668368
```

```
emp.columns
```

```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

```
emp.shape
```

```
(6, 6)
```

```
emp.head()
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year

```
emp.tail()
```

	Name	Domain	Age	Location	Salary	Exp
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

```
emp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       4 non-null      object
 3   Location  4 non-null      object
 4   Salary    6 non-null      object
 5   Exp       5 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

```
emp
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

```
emp.isnull()
```

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

```
emp.isna()
```

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

```
emp.isnull().sum()
```

```
Name        0
Domain      0
Age         2
Location    2
Salary      0
Exp         1
dtype: int64
```

```
emp.columns
```

```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

```
emp.dtypes
```

```
Name        object
Domain      object
Age         object
Location    object
Salary      object
Exp         object
dtype: object
```

```
emp['Name']
```

```
0      Mike
1    Teddy^
2     Uma#r
3      Jane
```

```
4    Uttam*
5       Kim
Name: Name, dtype: object
```

```
emp.describe()
```

	Name	Domain	Age	Location	Salary	Exp
count	6	6	4	4	6	5
unique	6	6	4	4	6	5
top	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
freq	1	1	1	1	1	1

Cleaning the data of Categorial coloumn

```
emp['Name'] = emp['Name'].str.replace(r'\W', '', regex = True)
```

```
emp
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy	Testing	45' yr	Bangalore	10%%000	<3
2	Umar	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

```
emp['Domain'] = emp['Domain'].str.replace(r'\W', '', regex = True)
```

```
emp['Domain']
```

```
0    Datascience
1        Testing
2    Dataanalyst
3      Analytics
4     Statistics
5            NLP
Name: Domain, dtype: object
```

```
emp['Location'] = emp['Location'].str.replace(r'\W', '' , regex = True)
```

```
emp['Location']
```

```
0       Mumbai
1    Bangalore
2          NaN
3     Hyderbad
4          NaN
5        Delhi
Name: Location, dtype: object
```

```
emp['Age'] = emp['Age'].str.extract(r'(\d+)') # d+ is digit match digit only.
```

```
emp['Age']
```

```
0     34
1     45
2    NaN
3    NaN
4     67
5     55
Name: Age, dtype: object
```

```
emp
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5^00#0	2+
1	Teddy	Testing	45	Bangalore	10%%000	<3
2	Umar	Dataanalyst	NaN	NaN	1$5%000	4> yrs
3	Jane	Analytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67	NaN	30000-	5+ year
5	Kim	NLP	55	Delhi	6000^$0	10+

#Cleaning of Numerical Data

```
emp['Salary'] = emp['Salary'].str.replace(r'\W', '', regex = True)
```

```
emp['Salary']
```

```
0     5000
1    10000
2    15000
3    20000
4    30000
5    60000
Name: Salary, dtype: object
```

```
emp['Exp'] = emp['Exp'].str.extract(r'(\d+)') # d+ is digit match digit only.
```

```
emp['Exp']
```

```
0      2
1      3
2      4
3    NaN
4      5
5     10
Name: Exp, dtype: object
```

```
emp
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

Missing value treatment

```
import numpy as np
```

```
clean_data = emp.copy() # Here created copy of clean data
clean_data
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
clean_data.isna().sum()
```

```
Name        0
Domain      0
Age         2
Location    2
Salary      0
Exp         1
dtype: int64
```

```
clean_data['Age'] =
         clean_data['Age'].fillna(np.mean(pd.to_numeric(clean_data['Age'])))
```

```
clean_data['Age']
```

```
0       34
1       45
2    50.25
3    50.25
4       67
5       55
Name: Age, dtype: object
```

```
clean_data['Location'] =
         clean_data['Location'].fillna(clean_data['Location'].mode()[0])
```

```
clean_data['Location']
```

```
0       Mumbai
1    Bangalore
2    Bangalore
3     Hyderbad
4    Bangalore
5        Delhi
Name: Location, dtype: object
```

```
clean_data['Exp'] =
         clean_data['Exp'].fillna(np.mean(pd.to_numeric(clean_data['Exp'])))
```

```
clean_data['Exp']
```

```
0      2
1      3
2      4
3    4.8
4      5
5     10
Name: Exp, dtype: object
```

```
clean_data
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50.25	Bangalore	15000	4
3	Jane	Analytics	50.25	Hyderbad	20000	4.8
4	Uttam	Statistics	67	Bangalore	30000	5
5	Kim	NLP	55	Delhi	60000	10

Now changing Datatypes as per coloumn names. To Do So we use ASTYPE: Convert system inbuilt data type to user define datatype.

```
type(clean_data)
```

```
pandas.core.frame.DataFrame
```

```
id(clean_data)
```

```
1682314377728
```

```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6 non-null      object
```

```
 1   Domain    6 non-null      object
 2   Age       6 non-null      object
 3   Location  6 non-null      object
 4   Salary    6 non-null      object
 5   Exp       6 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

```
clean_data.columns
```

```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

```
clean_data['Age'] = clean_data['Age'].astype(int)
clean_data['Exp'] = clean_data['Exp'].astype(int)
clean_data['Salary'] = clean_data['Salary'].astype(int)

clean_data['Name'] = clean_data['Name'].astype('category')
clean_data['Domain'] = clean_data['Domain'].astype('category')
clean_data['Location'] = clean_data['Location'].astype('category')

clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6 non-null      category
 1   Domain    6 non-null      category
 2   Age       6 non-null      int64
 3   Location  6 non-null      category
 4   Salary    6 non-null      int64
 5   Exp       6 non-null      int64
dtypes: category(3), int64(3)
memory usage: 938.0 bytes
```

```
clean_data
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Bangalore	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	Bangalore	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
clean_data.to_csv('clean_data.csv')

import os
os.getcwd()
```

```
'C:\\Users\\HP'
```